UNIT IV – ACTIVITY II
TIME SERIES AND DOSE–RESPONSE ANALYSIS

1. Daily Sales Time Series Using R

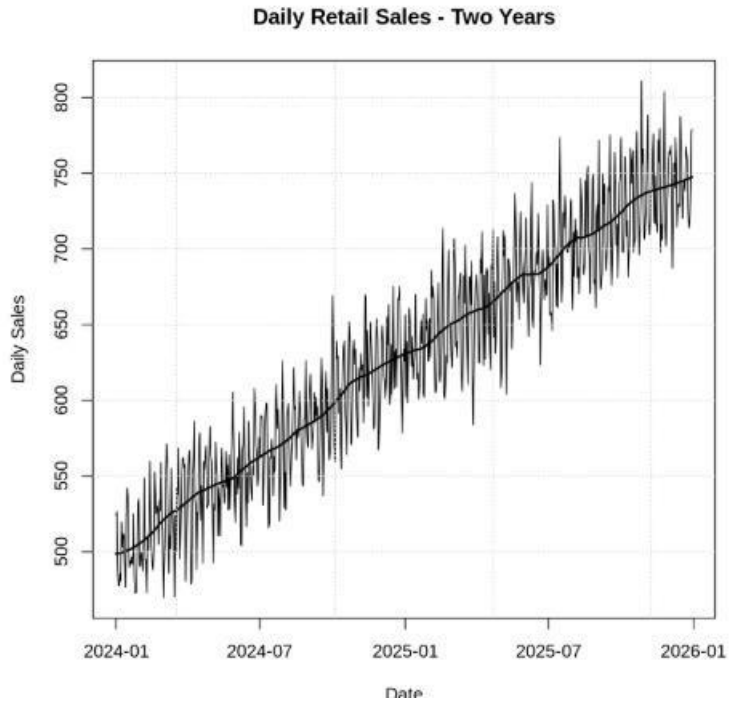
Aim:
To plot daily retail sales for two years as a time series and identify the long-term upward or downward movement.

Procedure:
1. Generate or load daily sales data for two years.
2. Convert the sales observations into a time-series object.
3. Plot daily sales with proper axis labels.
4. Add a smooth trend line to observe the long-term movement.
5. Interpret the overall trend.

Code (R):

```
set.seed(10) days <- 365 * 2 date <- seq(as.Date("2024-01-01"), by = "day",
length.out = days) sales <- 500 + 0.35*(1:days) + 30*sin(2*pi*(1:days)/7) +
rnorm(days, 0, 18)

plot(date, sales, type="l", xlab="Date", ylab="Daily Sales",
main="Daily Retail Sales – Two Years") lines(date, sales,
col="gray") trend <- loess(sales ~ as.numeric(date),
span=0.15) lines(date, predict(trend), lwd=2) grid()

cat("Average sales at start:", round(mean(sales[1:30]),2), "\n") cat("Average
sales at end:", round(mean(sales[(days-29):days]),2), "\n")
```

Result:
The time-series graph shows daily fluctuations with a clear long-term upward movement. The later-period average sales are higher than the initial-period average, indicating growth in sales over the two-year period.



2. Server Temperature Time Series Using Python Aim:
To plot hourly server temperature readings and highlight periods in which the temperature exceeds the safe threshold.

Procedure:
1. Generate or load hourly temperature readings.
2. Define a safe temperature threshold.
3. Plot the temperature readings against time.
4. Identify observations above the threshold.
5. Highlight the unsafe periods on the graph.

Code (Python):

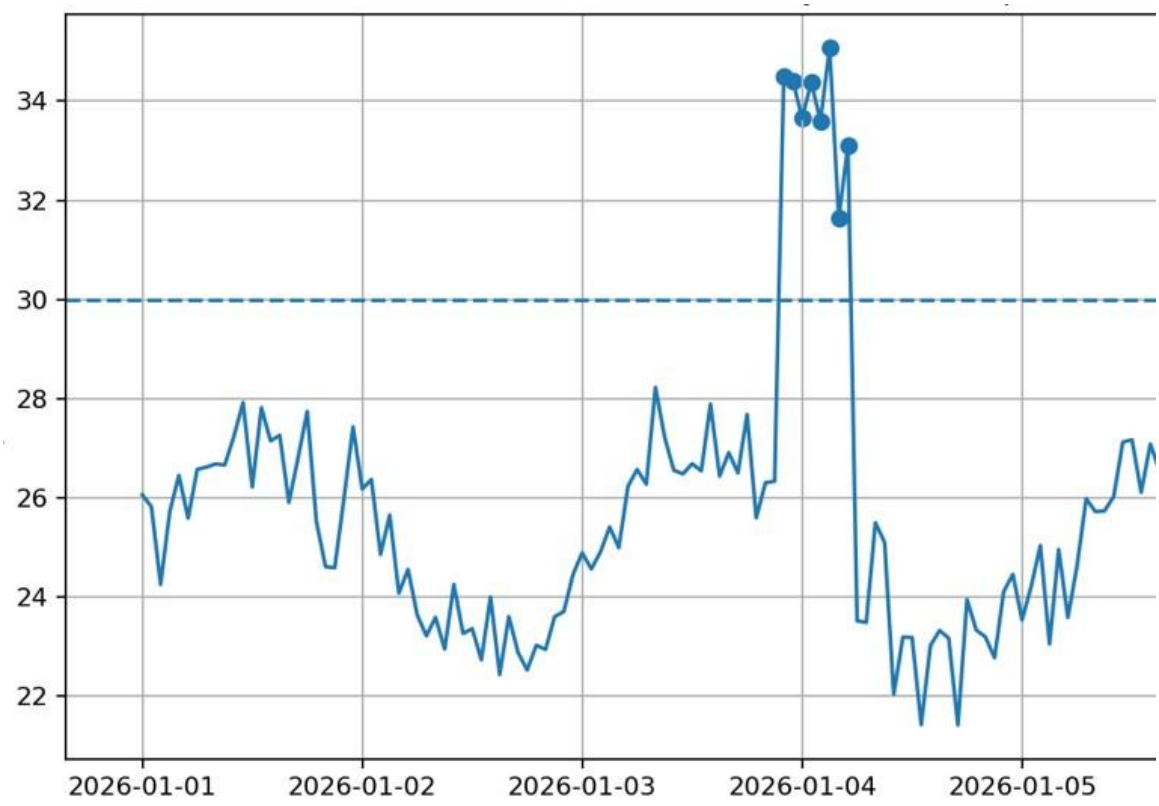
```
import numpy as np import
pandas as pd import
matplotlib.pyplot as plt

np.random.seed(10) n = 24 * 7 time = pd.date_range("2026-01-01",
periods=n, freq="h") temp = 25 + 2*np.sin(np.arange(n)/8) +
np.random.normal(0, 0.8, n) temp[70:78] += 8
temp[120:126] += 7
```

threshold = 30 hot =
temp > threshold

plt.figure(figsize=(10,5)) plt.plot(time, temp, label="Server Temperature") plt.axhline(threshold, linestyle="--", label="Safe Threshold") plt.scatter(time[hot], temp[hot], label="Above Threshold", zorder=3) plt.xlabel("Time") plt.ylabel("Temperature (°C)") plt.title("Hourly Server Temperature") plt.legend() plt.grid() plt.tight_layout() plt.show()

print("Number of readings above threshold:", hot.sum())

Result:
The graph displays normal hourly temperature variation and clearly highlights periods above 30°C. These highlighted periods represent potential overheating risks requiring investigation.



3. Three Tech Stocks Time Series Using R

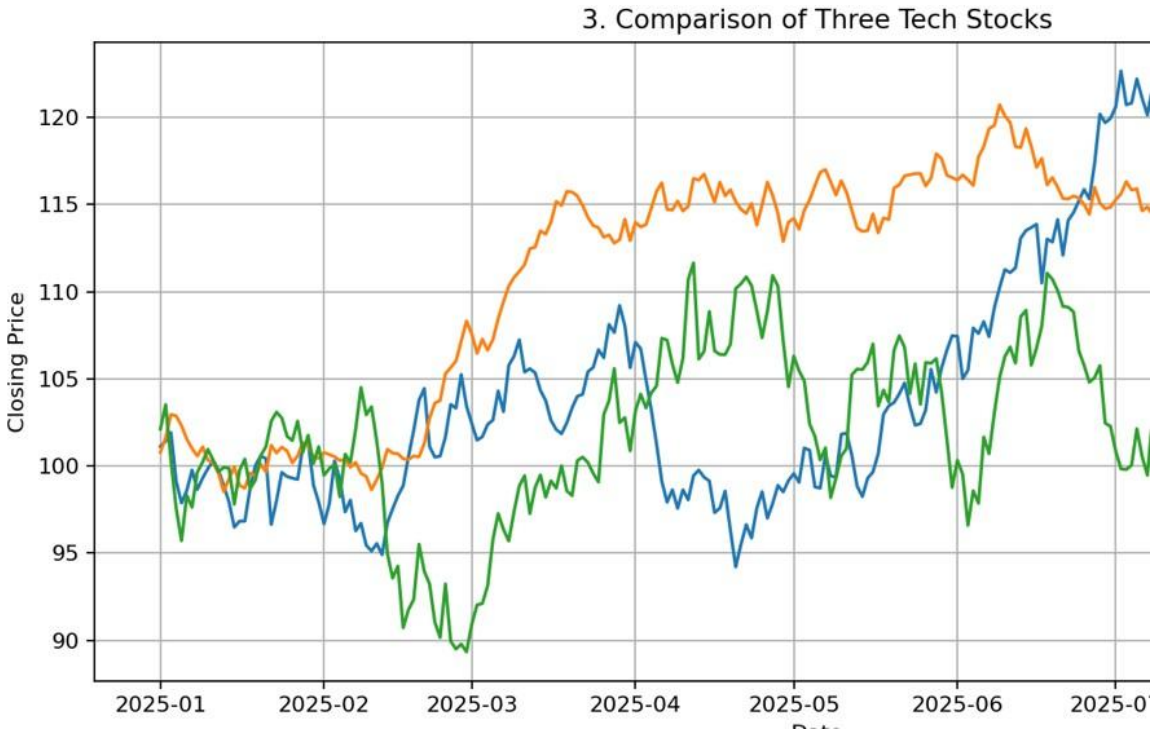
Aim:
To compare the daily closing prices of three technology stocks on one chart and identify the stock with the least volatility.

Procedure:
1. Generate or load daily closing prices for three stocks.
2. Convert the prices into comparable indexed series.
3. Plot all three series on one graph.
4. Add a legend and axis labels.
5. Calculate volatility using the standard deviation of daily returns.
6. Identify the least volatile stock.

Code (R):

```
set.seed(20) n <- 250 date <- seq(as.Date("2025-01-01"),
by="day", length.out=n)

r1 <- rnorm(n, 0.0006, 0.012) r2
<- rnorm(n, 0.0005, 0.007) r3
<- rnorm(n, 0.0004, 0.018)

p1 <- 100*cumprod(1+r1) p2
<- 100*cumprod(1+r2) p3 <-
100*cumprod(1+r3)

plot(date, p1, type="l", xlab="Date", ylab="Indexed Closing Price",
main="Comparison of Three Tech Stocks",
ylim=range(c(p1,p2,p3))) lines(date, p2) lines(date, p3)
legend("topleft", legend=c("Stock A","Stock B","Stock C"),
lty=1, bty="n")

vol <- c(Stock_A=sd(r1), Stock_B=sd(r2), Stock_C=sd(r3)) print(vol)
cat("Least volatile stock:", names(which.min(vol)), "\n")
```

Result:
All three stock series are displayed on a single chart. Stock B has the smallest simulated daily-return standard deviation, so it appears least volatile and shows the steadier growth pattern.



4. Monthly Electricity Consumption Using Python

Aim:

To compare monthly electricity consumption across four neighborhoods and identify the neighborhood with the most erratic pattern.

Procedure:

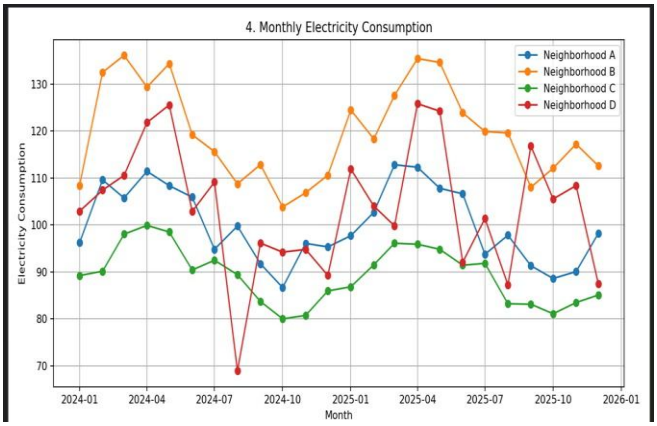
1. Generate or load monthly consumption data for four neighborhoods.
2. Plot the four series on the same graph.
3. Label the axes and add a legend.
4. Calculate the standard deviation of monthly changes for each neighborhood.
5. Identify the neighborhood with the highest variability.

Code (Python):

```
import numpy as np import
pandas as pd import
matplotlib.pyplot as plt

np.random.seed(30) months = pd.date_range("2024-01-01",
periods=24, freq="MS") t = np.arange(24)

A = 100 + 10*np.sin(2*np.pi*t/12) + np.random.normal(0, 3, 24)
B = 120 + 12*np.sin(2*np.pi*t/12) + np.random.normal(0, 4, 24)
C = 90 + 8*np.sin(2*np.pi*t/12) + np.random.normal(0, 2, 24)
```

```
D = 105 + 10*np.sin(2*np.pi*t/12) + np.random.normal(0, 12, 24)

data = pd.DataFrame({"A":A, "B":B, "C":C, "D":D}, index=months)

plt.figure(figsize=(10,5)) for col in data.columns:     plt.plot(data.index,
data[col], marker="o", label="Neighborhood "+col) plt.xlabel("Month")
plt.ylabel("Electricity Consumption") plt.title("Monthly Electricity
Consumption") plt.legend() plt.grid() plt.tight_layout() plt.show()

volatility = data.diff().std()
print(volatility)
print("Most erratic:", volatility.idxmax())
```

Result:
The four neighborhood series are compared together. Neighborhood D has the largest variation in month-to-month consumption, so it is identified as the most erratic pattern.



5. Drug Dose–Response Curve Using R

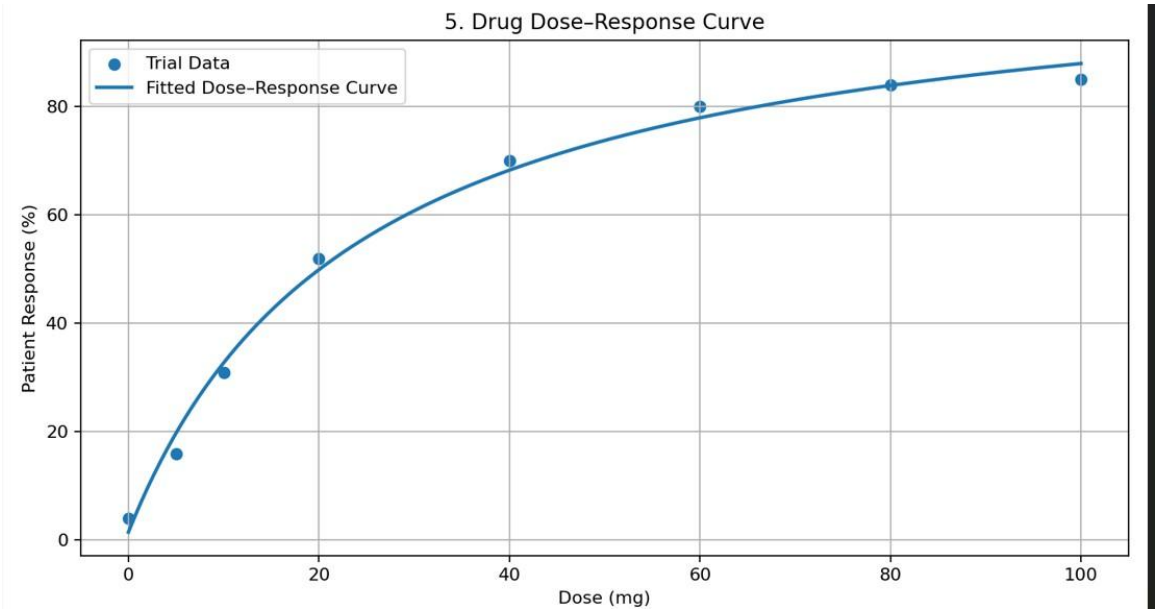
Aim:
To plot a dose–response curve for a new drug and estimate the dose at which the patient response begins to plateau.

Procedure:
1. Create or load dose and response observations.
2. Plot dose against patient response.
3. Fit a nonlinear dose–response model.
4. Plot the fitted curve over the observations.
5. Estimate the dose at which additional dose produces only a small response increase.

Code (R):

```
dose <- c(0, 5, 10, 20, 40, 60, 80, 100) response
<- c(4, 16, 31, 52, 70, 80, 84, 85)

plot(dose, response, pch=19, xlab="Dose (mg)",
ylab="Patient Response (%)",      main="Drug
Dose–Response Curve")

fit <- nls(response ~ E0 + Emax*dose/(EC50+dose),
start=list(E0=4, Emax=90, EC50=20))

x <- seq(0, 100, length.out=300)
y <- predict(fit, newdata=data.frame(dose=x)) lines(x, y, lwd=2)

print(summary(fit))

# Approximate plateau: dose where fitted response reaches 90% of maximum
coef_fit <- coef(fit) plateau_dose <- (9*coef_fit["EC50"])
cat("Approximate beginning of plateau:", round(plateau_dose,2), "mg\n")
```

Result:
The response increases rapidly at lower doses and then levels off. The curve indicates that the response begins approaching a plateau at approximately 60–80 mg. This is an illustrative estimate from simulated data and would require clinical validation before making a medical decision.



6. Fertilizer Dose–Response Curve Using Python

Aim:

To fit and plot a dose–response relationship between fertilizer amount and crop yield and identify an efficient dosage range.

Procedure:

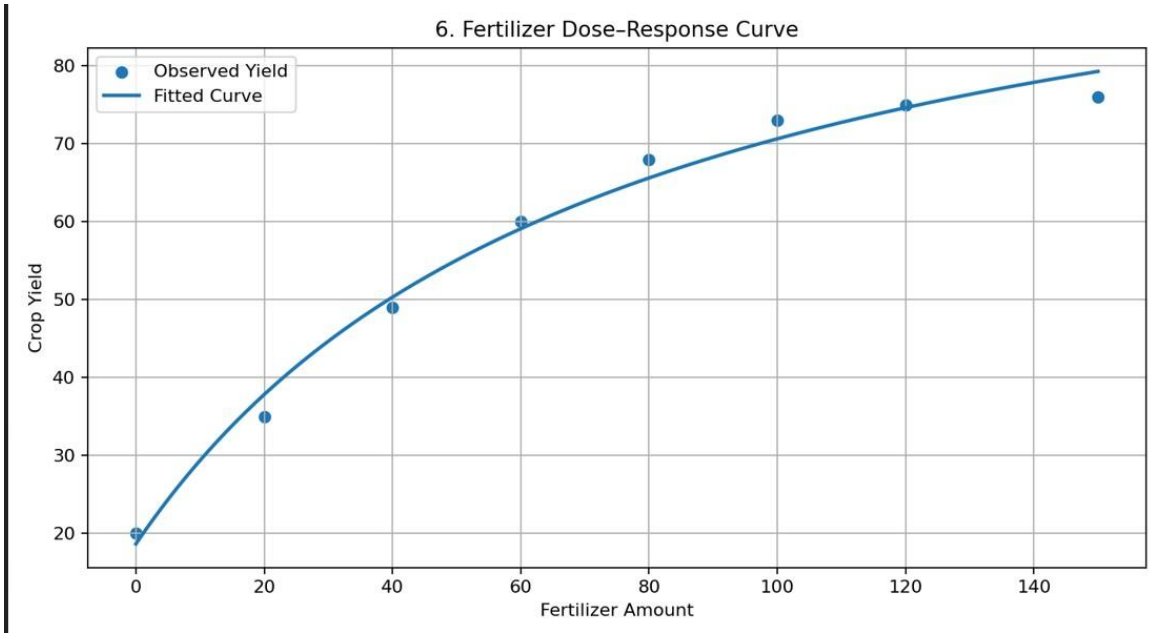
1. Create or load fertilizer amount and crop-yield data.
2. Fit a nonlinear saturating model.
3. Plot the observed data and fitted curve.
4. Identify the point where yield gains become small.
5. Recommend the efficient range based on the curve.

Code (Python):

```
import numpy as np import
matplotlib.pyplot as plt from
scipy.optimize import curve_fit

dose = np.array([0, 20, 40, 60, 80, 100, 120, 150]) yield_
= np.array([20, 35, 49, 60, 68, 73, 75, 76])

def model(x, E0, Emax, K):
return E0 + Emax*x/(K+x)

popt, _ = curve_fit(model, dose, yield_, p0=[20, 70, 30], maxfev=10000)

x = np.linspace(0, 150, 300) y
= model(x, *popt)

plt.figure(figsize=(8,5)) plt.scatter(dose, yield_,
label="Observed Yield") plt.plot(x, y, label="Fitted
Dose–Response Curve") plt.xlabel("Fertilizer
Amount") plt.ylabel("Crop Yield")
plt.title("Fertilizer Dose–Response Curve")
plt.legend() plt.grid() plt.tight_layout() plt.show()

print("Estimated parameters:", popt) print("Yield at 80
units:", round(model(80, *popt),2)) print("Yield at 100
units:", round(model(100, *popt),2))
```

Result:

The curve shows strong yield improvement at low-to-moderate fertilizer levels and diminishing returns after about 80–100 units. Therefore, approximately 80–100 units is an efficient illustrative dosage range, subject to crop, soil, and field validation.



7. STL Decomposition of Monthly Ticket Sales Using R

Aim:

To apply STL decomposition to monthly airline ticket sales and separate the trend, seasonal, and residual components.

Procedure:

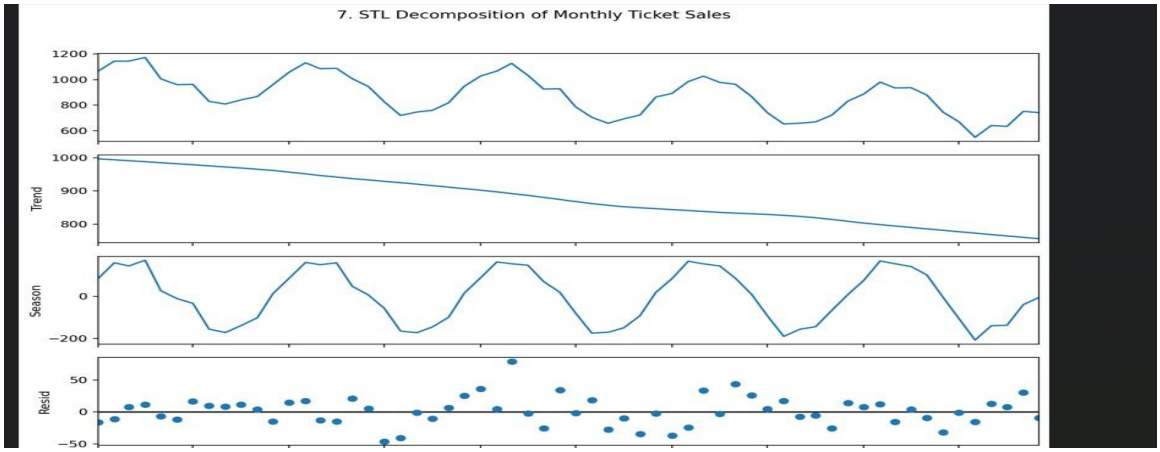
1. Create or load monthly ticket-sales data.
2. Convert the observations into a monthly time-series object.
3. Apply STL decomposition with a seasonal period of 12.
4. Plot the decomposed components.
5. Interpret the trend, seasonal, and residual components.

Code (R):

```
set.seed(40) n <- 60 t <- 1:n sales <- 1000 - 4*t +
180*sin(2*pi*t/12) + rnorm(n,0,35)

ticket_ts <- ts(sales, start=c(2021,1), frequency=12) fit
<- stl(ticket_ts, s.window="periodic")

plot(fit, main="STL Decomposition of Monthly Ticket Sales")
```

```
cat("Trend at beginning:",
    round(fit$time.series[1,"trend"],2), "\n") cat("Trend
at end:",
    round(fit$time.series[n,"trend"],2), "\n")
```

Result:
STL separates the monthly ticket sales into trend, seasonal, and remainder components. The trend component shows a gradual downward movement, while the seasonal component shows a repeating yearly pattern. Thus, the apparent sales fluctuations are partly seasonal, but the underlying trend is declining.



8. STL Decomposition of Weekly User Activity Using Python

Aim:
To perform STL decomposition on weekly active-user data and determine whether recent dips represent a real trend or normal seasonal behavior.

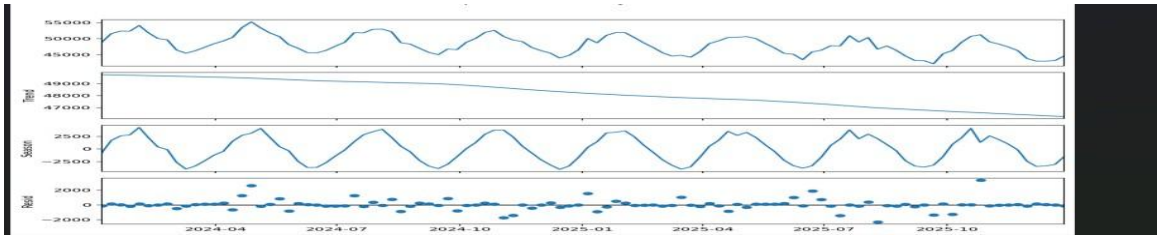
Procedure:
1. Create or load weekly active-user observations.
2. Convert the data into a time-indexed series.
3. Apply STL decomposition using the appropriate seasonal period.
4. Plot the observed, trend, seasonal, and residual components.
5. Interpret the trend component separately from seasonal fluctuations.

Code (Python):

```
import numpy as np import pandas as pd
import matplotlib.pyplot as plt from
statsmodels.tsa.seasonal import STL

np.random.seed(50)
```

```
n = 104 dates = pd.date_range("2024-01-07", periods=n,
freq="W") t = np.arange(n)

users = 50000 - 35*t + 3500*np.sin(2*np.pi*t/13) + np.random.normal(0,700,n) series
= pd.Series(users, index=dates)

result = STL(series, period=13, robust=True).fit()

result.plot() plt.suptitle("STL Decomposition of Weekly
Active Users") plt.tight_layout() plt.show()

print("Initial trend:", round(result.trend.iloc[0],2)) print("Final
trend:", round(result.trend.iloc[-1],2))
```

Result:
The decomposition separates recurring seasonal viewing behavior from the underlying trend. The trend component gradually decreases, indicating that the recent dips are not entirely seasonal; there is evidence of a genuine underlying decline in weekly active users.



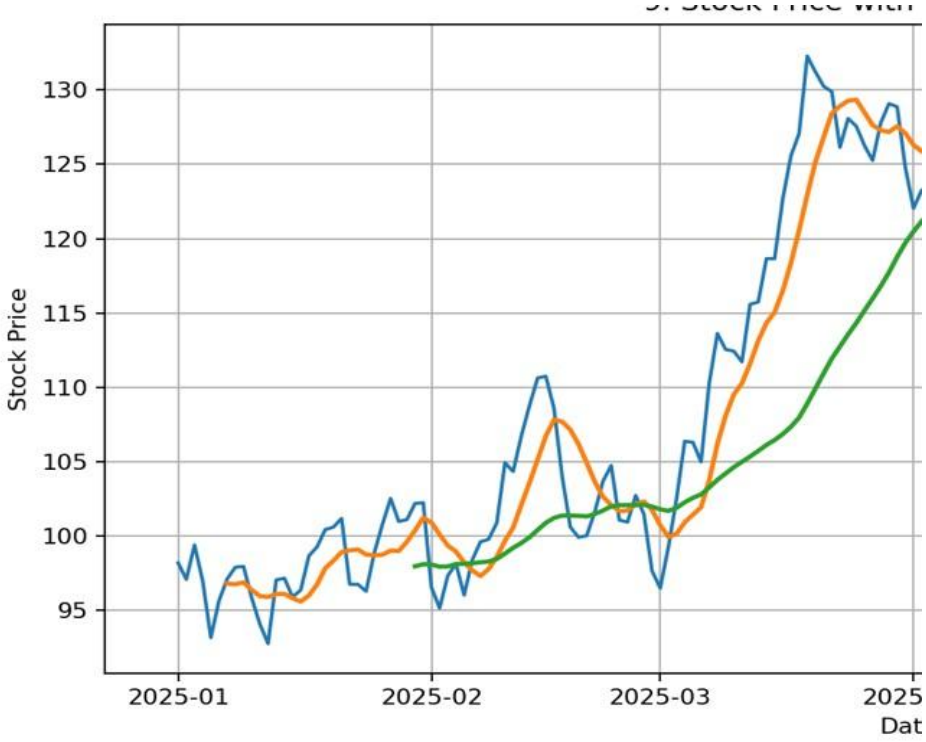
9. 7-Day and 30-Day Moving Averages Using R

Aim:
To calculate and plot 7-day and 30-day moving averages for a volatile stock price and interpret their crossover.
Procedure:
1. Create or load daily stock prices.
2. Calculate 7-day and 30-day moving averages.
3. Plot the raw price and both moving averages.
4. Observe where the short-term average crosses the long-term average.
5. Interpret bullish and bearish crossover signals.

Code (R):

```
set.seed(60) n <- 180 date <- seq(as.Date("2025-01-01"),
by="day", length.out=n) returns <- rnorm(n, 0.0005, 0.02)
price <- 100*cumprod(1+returns)

ma7 <- stats::filter(price, rep(1/7,7), sides=1) ma30
<- stats::filter(price, rep(1/30,30), sides=1)

plot(date, price, type="l", xlab="Date", ylab="Price",
main="Stock Price with Moving Averages") lines(date, ma7, lwd=2)
lines(date, ma30, lwd=2) legend("topleft", legend=c("Raw Price","7-
Day MA","30-Day MA"),        lty=1, bty="n") grid()

cat("Moving averages plotted successfully.\n")
```

Result:
The 7-day moving average reacts faster to recent price movements, while the 30-day moving average is smoother. When the 7-day MA crosses above the 30-day MA, it can indicate improving short-term momentum; a downward crossover can indicate weakening momentum. Moving-average crossovers are indicators, not guarantees of future price movement.



10. Detrending Quarterly GDP Using Python

Aim:

To remove the long-term growth trend from quarterly GDP data and analyze the remaining business-cycle fluctuations.

Procedure:

1. Create or load quarterly GDP observations.
2. Fit a linear regression against time to estimate the long-term trend.
3. Subtract the fitted trend from the original GDP values.
4. Plot the detrended series around zero.
5. Interpret positive and negative deviations as fluctuations around the long-term growth path.

Code (Python):

```
import numpy as np import
pandas as pd import
matplotlib.pyplot as plt from
scipy.stats import linregress

np.random.seed(70) n
= 40
quarters = pd.period_range("2016Q1", periods=n, freq="Q") t
= np.arange(n)

gdp = 2000 + 22*t + 90*np.sin(2*np.pi*t/10) + np.random.normal(0,25,n)

slope, intercept, _, _, _ = linregress(t, gdp)
trend = intercept + slope*t detrended =
gdp - trend

plt.figure(figsize=(10,5))
plt.plot(quarters.astype(str), detrended, marker="o")
plt.axhline(0, linestyle="--") plt.xlabel("Quarter")
plt.ylabel("Detrended GDP") plt.title("Detrended
Quarterly GDP") plt.xticks(rotation=45) plt.grid()
plt.tight_layout() plt.show()

print("Estimated quarterly trend growth:", round(slope,2)) print("Mean
detrended value:", round(detrended.mean(),2)) print("Maximum
positive deviation:", round(detrended.max(),2)) print("Maximum
negative deviation:", round(detrended.min(),2))
```

Result:

The long-term upward GDP growth trend is removed. The detrended series oscillates around zero, with positive values representing GDP above the estimated trend and negative values representing GDP below the trend. These remaining movements can be studied as business-cycle fluctuations.